

GCI

Feature Engineering



Overview of Feature Engineering

Transforming Numerical Variables

Transforming Categorical Variables

Use of Domain Knowledge

Ensembling

Overview of Feature Engineering

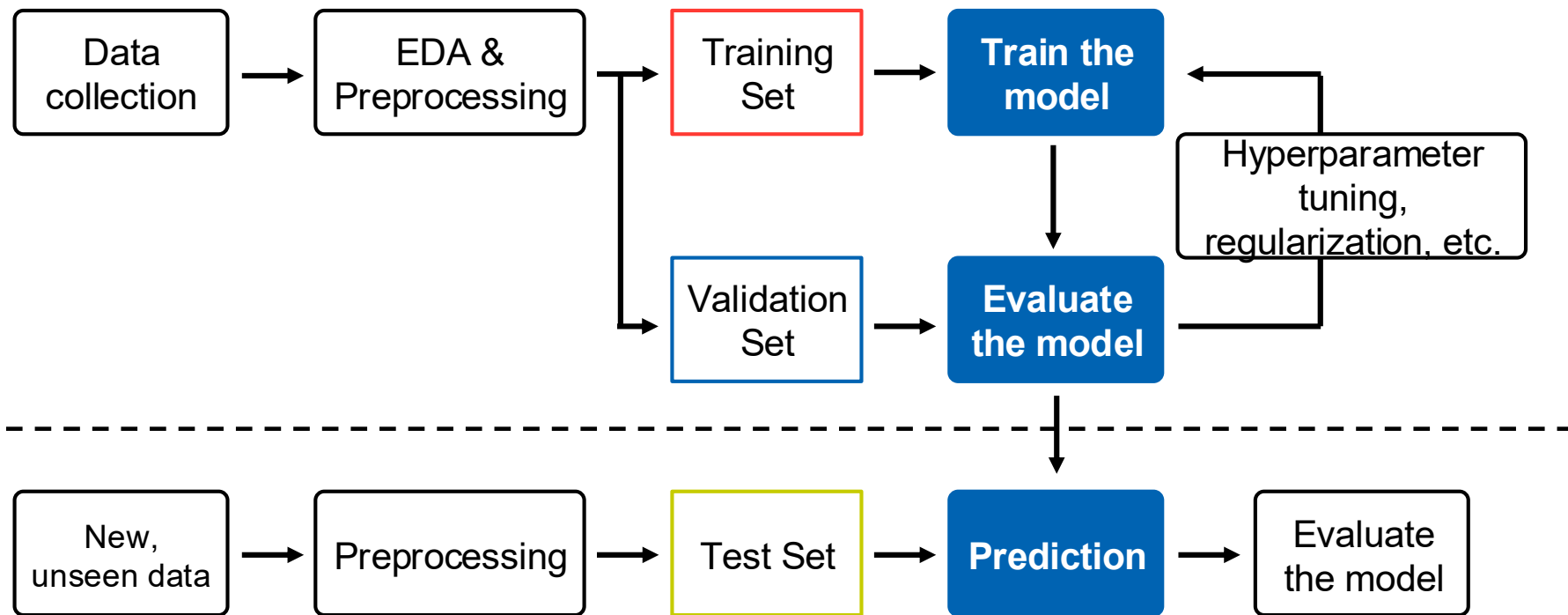
Transforming Numerical Variables

Transforming Categorical Variables

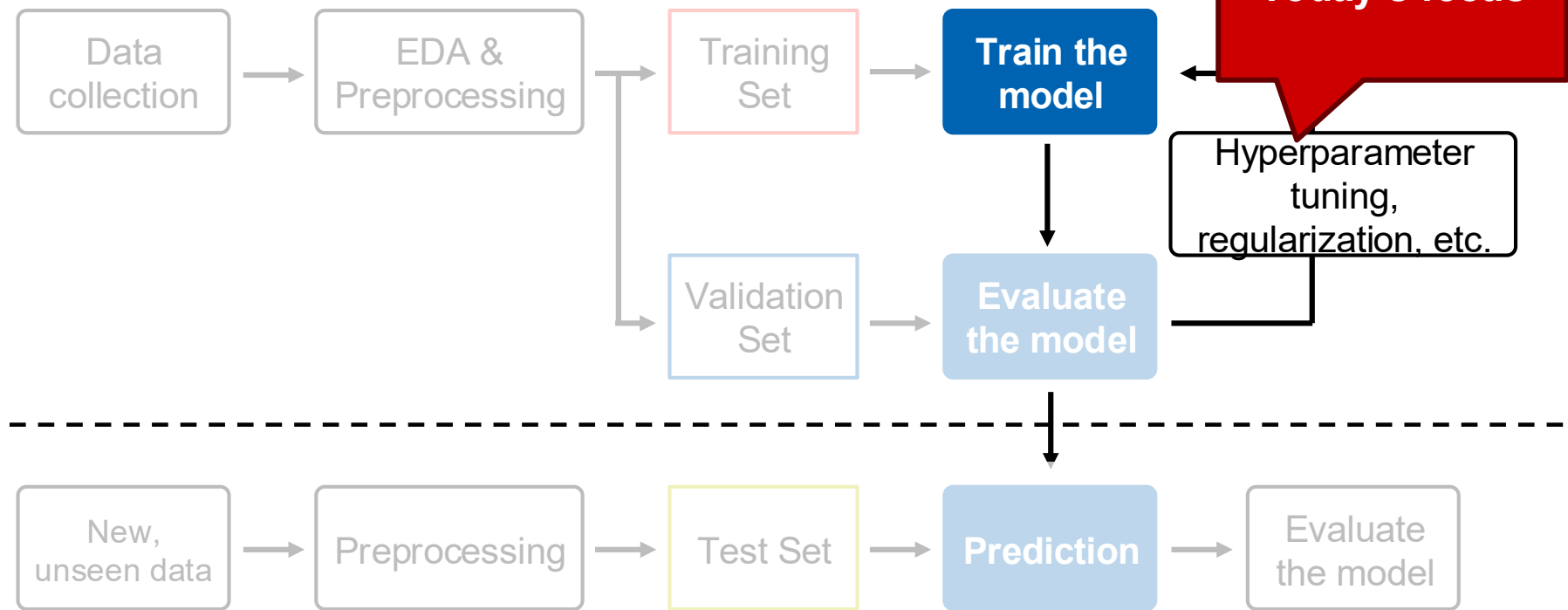
Use of Domain Knowledge

Ensembling

Supervised Learning Workflow (Summary)

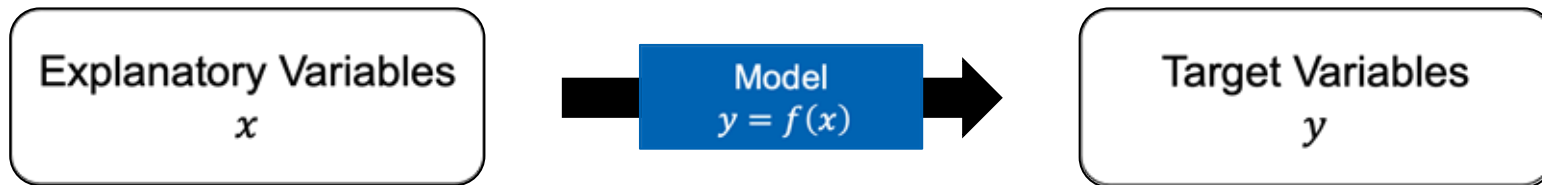


Supervised Learning Workflow (Summary)



Feature Engineering

The process selecting, transforming, or creating input variables to improve the performance of a machine learning model



e.g. Sales prediction

- ~~x: date, product ID~~ → **day, holiday or not, season, product category**
- y: sales

Why is feature engineering so important?

- To boost model performance
- To prevent overfitting
- To make your model more interpretable

Feature engineering often requires trial and error.

- In this lecture, we will learn general techniques for designing useful features for different datasets.



Transforming features

Transform the data for the model to handle inputs easier

- **Scaling, log-transformation**
- **Encoding**

Creating features

Make new features to improve model's performance

- **Cross/combine existing features**
- **Use domain knowledge to create new features**

Extracting features

Compress original information into important features of a small number

- Dimension extraction

Selecting features

Remove features when that the input dimension is too large than data points

- Feature selection

Overview of Feature Engineering

Transforming Numerical Variables

Transforming Categorical Variables

Use of Domain Knowledge

Ensembling

Classifications of variables



	Type	Description	Examples
Quantitative data (Numeric variables)	Ratio scale	Difference has meaning and has zero point	Body weight (50 kg vs 70kg), Distance (2km vs 200km)
	Interval scale	Difference has meaning	Temperature (15°C, 30°C), Year (2000, 2025)
Qualitative data (Categorical variables)	Ordinal scale	Order has meaning	Academic score (A, B, C), questionnaire (good, bad)
	Nominal scale	Distinct meanings	Nationality (Japanese, Chinese), product ID (42, 57)

Overview of Feature Engineering

Transforming Numerical Variables

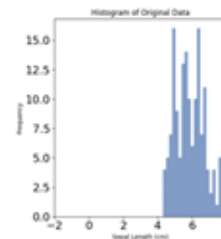
Transforming Categorical Variables

Use of Domain Knowledge

Ensembling

Scaling

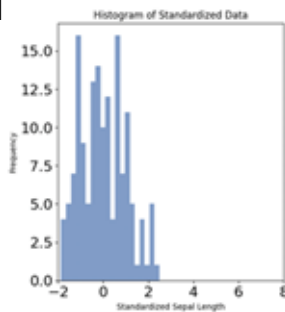
If features differ greatly in scale, the larger scaled features can have an undue influence on the results



- **Standardization**

Transform so that the mean is 0 and the standard deviation is 1

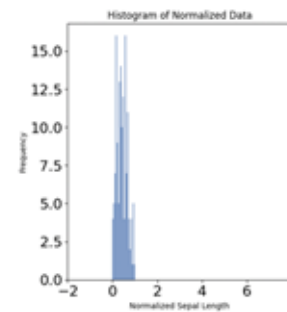
$$x_{std} = \frac{x - \overset{\text{Mean}}{\mu_x}}{\underset{\text{Standard Deviation}}{\sigma_x}}$$



- **Normalization**

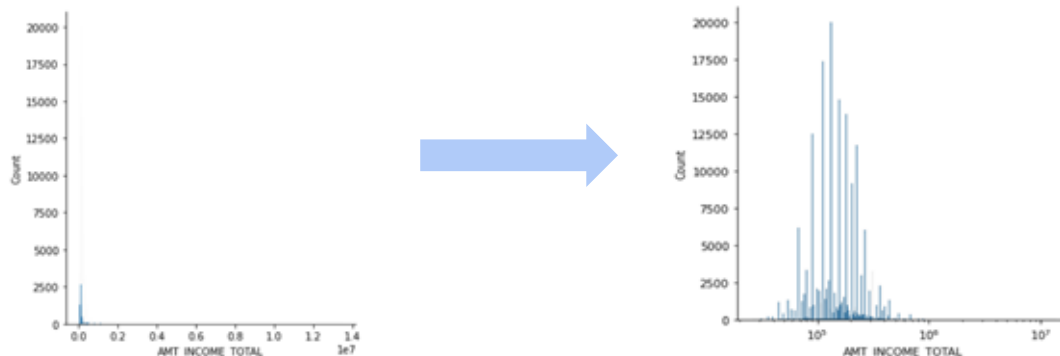
Transform so that the minimum is 0 and the maximum is 1

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$



Log transformation

- Compresses large values and makes relationships between variables easier for a model to capture
- Useful when dealing with heavily skewed data
- log is not defined for zeros or negative values



Standardization, normalization, log transformation

```
import numpy as np

# standardization
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
x_std = ss.fit_transform(x)

# normalization
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
x_normalized = scaler.fit_transform(x)

# log transform
x_log_trans = np.log(x)
```

Scaling has (almost) no effect on tree-based models

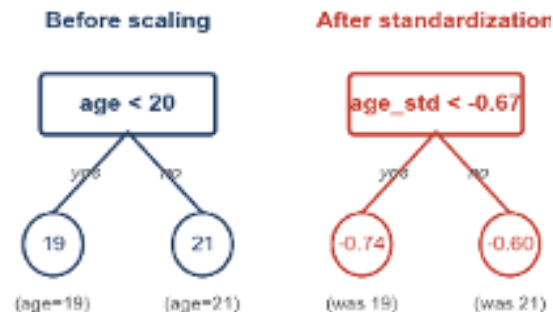


Scaling may not work in tree-based models

- Trees split on rank order — not absolute scale

Decision Tree / RF / XGBoost / LightGBM: results essentially identical

Still useful for: linear regression, logistic regression, k-NN, SVM



Overview of Feature Engineering

Transforming Numerical Variables

Transforming Categorical Variables

Use of Domain Knowledge

Ensembling

Label Encoding

Assign each category an integer (e.g. red→1, green→2, blue→3)

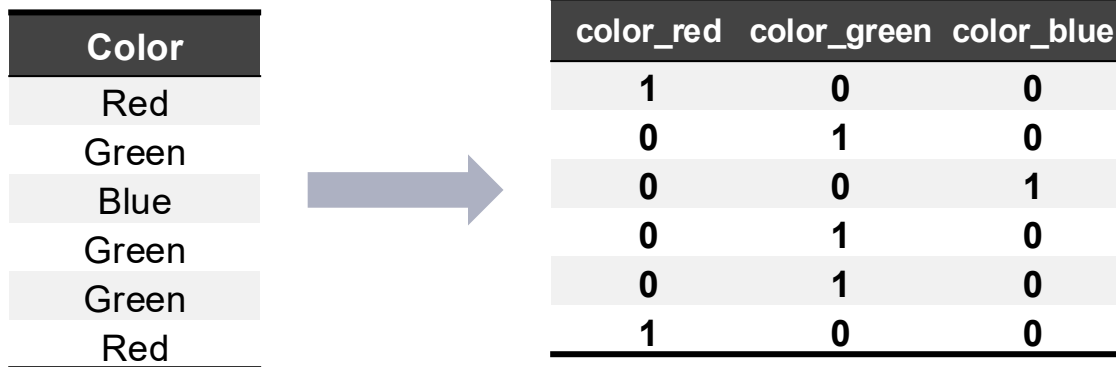
- Merits: Simple, Works fine with tree-based models
- Demerits: introduces a fake ordering

Color		Color
Red		1
Green		2
Blue		3
Green		2
Green		2
Red		1

One-hot Encoding

Create a separate column for each category and assign 1 to the matched category

- Merits: removes numerical relationships
- Demerits: high memory consumption



Color	color_red	color_green	color_blue
Red	1	0	0
Green	0	1	0
Blue	0	0	1
Green	0	1	0
Green	0	1	0
Red	1	0	0

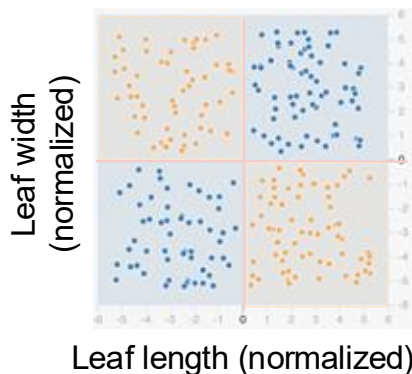
Cross features

A new feature built from a product of 2+ existing ones

- $a, b, c \rightarrow a \times b, a \times c, b \times c, a \times b \times c \dots$
- e.g., add (room size) $\times$ (distance to station) in rent prediction

Merits: captures interactions the model cannot easily learn on its own

Demerits: memory consumption, overfitting risk



Length	Width	Class
1	1	0
3	4	0
-3	-2	0
-2	1	1
4	-4	1



Length x Width	Class
1	0
12	0
6	0
-2	1
-16	1

Label / one-hot encoding & cross features

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# sample categorical data
df = pd.DataFrame({
    "city": ["Tokyo", "Bangkok", "Tokyo", "Jakarta"],
    "device": ["mobile", "pc", "tablet", "mobile"]
})

# label encoding
df["city_label"] = LabelEncoder().fit_transform(df["city"])

# one-hot encoding
df_onehot = pd.get_dummies(df, columns=["city", "device"])
```

Label / one-hot encoding & cross features

```
import pandas as pd

# sample categorical data
df = pd.DataFrame({
    "city": ["Tokyo", "Bangkok", "Tokyo", "Jakarta"],
    "device": ["mobile", "pc", "tablet", "mobile"]
})

# cross feature
df["city_device"] = df["city"] + "_" + df["device"]
df_cross = pd.get_dummies(df, columns=["city_device"])
```

Overview of Feature Engineering

Transforming Numerical Variables

Transforming Categorical Variables

Use of Domain Knowledge

Ensembling

Domain knowledge (expertise in a specific field) helps us transform raw data into meaningful features that are more directly related to the prediction target.

e.g., Rent prediction

- Raw data
 - address, size, age of building
- Using domain knowledge, create features like
 - Distance to nearest station
 - Distance to nearest school



Domain Knowledge for Feature Engineering



Domain knowledge (expertise in a specific field) helps us transform raw data into meaningful features that are more directly related to the prediction target.

Domain/task	Raw data	Domain knowledge-based features
 Real estate: predict house prices	Address, size, age of building	Distance to nearest station, price per square meter, school district
 Soccer: predict whether a shot becomes a goal	Ball position, player positions	Distance to goal, shooting angle, number of defenders
 E-commerce: predict whether a user buys a product	Browsing history, purchase history	Number of views in the past week, time since last purchase

Overview of Feature Engineering

Transforming Numerical Variables

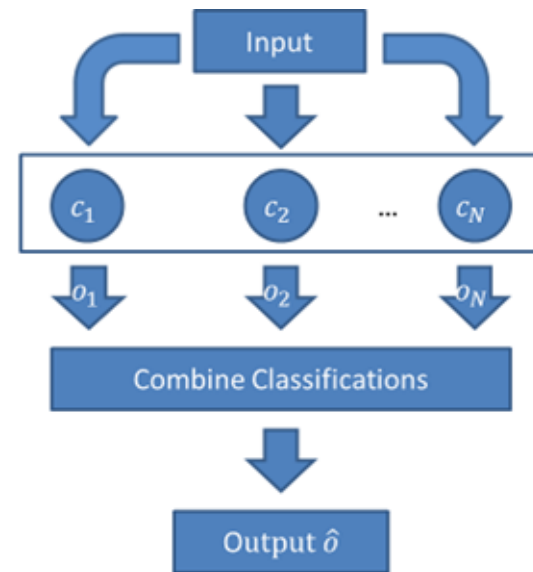
Transforming Categorical Variables

Use of Domain Knowledge

Ensembling

Ensemble learning

- Method to improve your model performance
- Idea: prepare multiple models, then aggregate result by majority vote or average
 - Majority tends to beat any single model
- Two main families: **Bagging** and **Boosting**

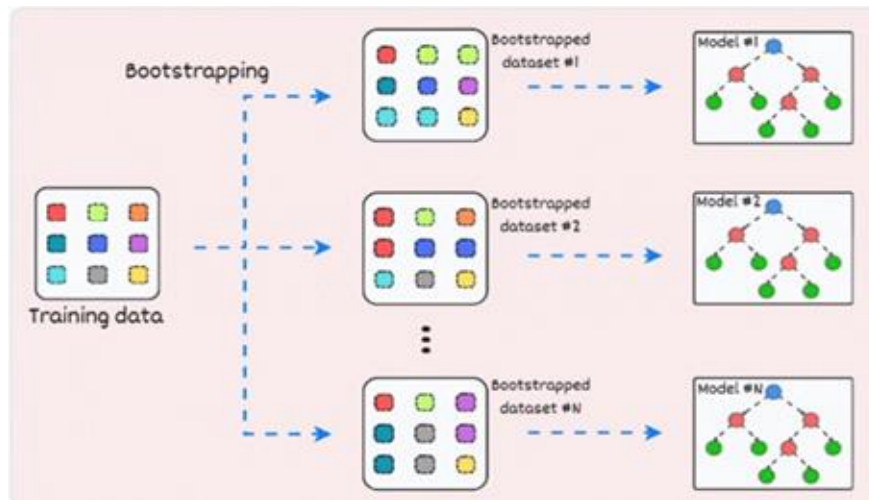


Reference:
<https://qiita.com/nirperm/items/318d7e210c059373f8d2>

Bagging (bootstrap aggregating)

Bagging

- Generate bootstrap resamples of training data
 - Bootstrap: to sample from the set with duplicates allowed
- Train separate model with each resample, then aggregate result
 - Each model may be weaker, but the combined model can be stronger

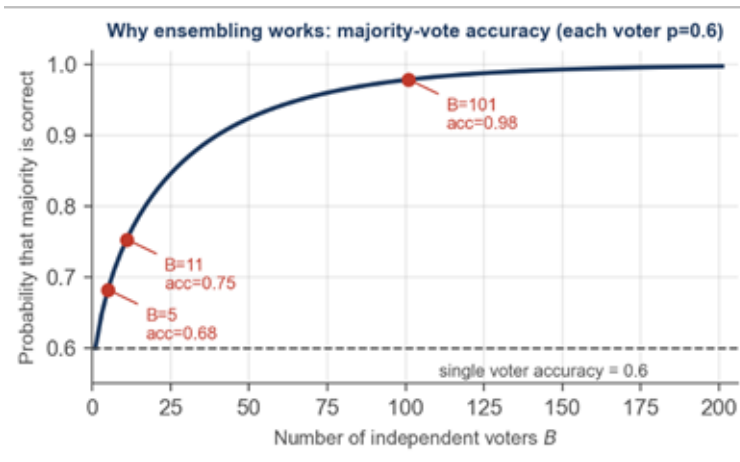


Why does bagging work?



100 imperfect models can outperform 1 strong model — if mistakes are diverse.

- 1 expert (60% correct) → 60% accuracy
- 5 independent voters → ~68%
- 11 voters → ~75%
- 101 voters → ~98%

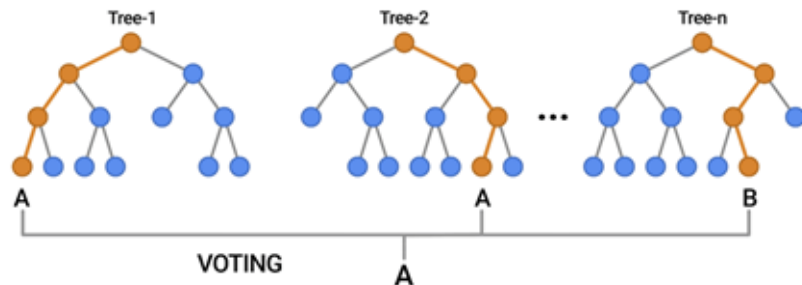


Caveat: if everyone makes the same mistake, voting doesn't work.

Bagging example: Random forest

Random forest

- Bagging used for decision trees
- Decision trees often look very similar (e.g. one strong feature always used for first split)
- Only use m features to decide each split
- Train many trees, then take the average



Reference:

<https://blog.tensorflow.org/2021/05/introducing-tensorflow-decision-forests.html>

Boosting

- Train a weak model and check incorrect predictions
- Reweight the data, train another model
- Repeat, then take the average

Examples: Adaboost, gradient boosting, XGBoost, lightGBM, etc.



<https://blog.dailydoseofds.com/p/an-animated-guide-to-bagging-and>

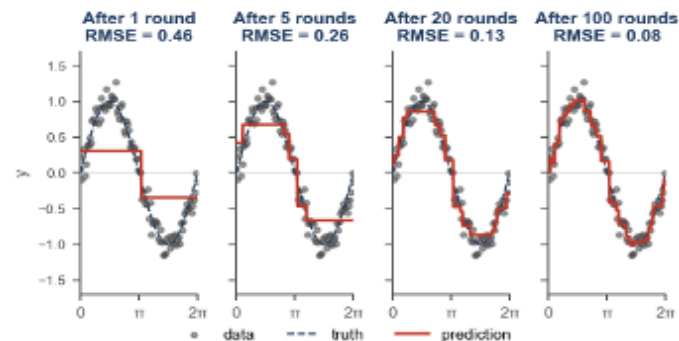
Why does boosting work?



Steps:

- Round 1 captures the easy patterns
- Round 2 fixes what's still wrong
- Round 3 fixes what's left after that
- ...

→ Residual (truth – prediction) shrinks every round



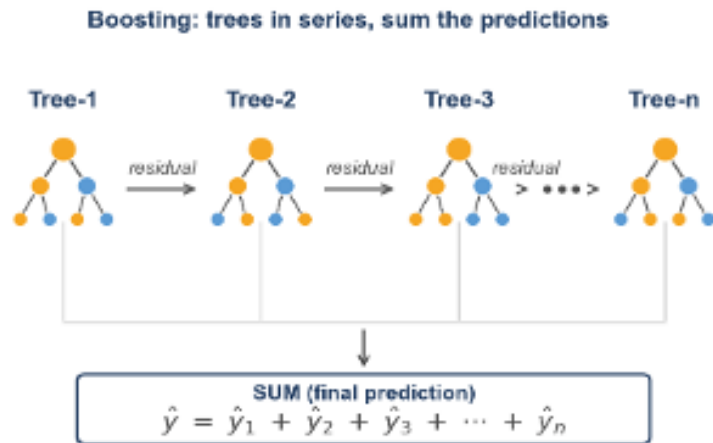
Caveat: try to decrease noises -> overfitting with increased iterations

Boosting example: XGBoost



XGBoost

- Boosting used for decision trees
- Each tree is shallow (depth $\approx 3-8$)
- Each new tree predicts the residual
- Train many trees, then sum (not average)



Bagging (Random forest)

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

base_clf = DecisionTreeClassifier(random_state=0)
clf = BaggingClassifier(
    base_estimator=base_clf, # <- classifier you want to apply bagging to
    n_estimators=100,        # number of classifiers
    random_state=0,          # for reproducibility
)

clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

Gradient boosting

```
from sklearn.ensemble import GradientBoostingRegressor

reg = GradientBoostingRegressor(
    random_state=0
)
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)
```

XGboosting

```
from xgboost import XGBRegressor

X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.2)

reg = XGBRegressor(
    n_estimators=1000,
    eval_metric='rmse',          # evaluation metric
    early_stopping_rounds=20     # finish learning to avoid overfitting
)

reg.fit(X_train, y_train,
        eval_set=[(X_valid, y_valid)])
y_pred = reg.predict(X_test)
```

Section summary



Bagging	RandomForest	When over-learning is to be avoided As a baseline model in the early stages of a project when useful features have not been identified	First option as bagging
	BaggingClassifier /Regressor		In applications where models except decision tree systems are suitable.
Boosting	XGBoost GradientBoosting	When you want to pursue accuracy When you have found useful features in EDA to some extent, etc.	First choice in many cases.
	AdaBoost		In applications where models except decision tree systems are suitable.
Ensemble not used	Stand-alone model	When interpretability of results matters	

Key Takeaways



- Feature engineering can impact your model's performance
 - Feature transformation: standardization, normalization, etc.
 - Feature creation: combine features, use domain knowledge, etc.

Key Takeaways



- Feature engineering can impact your model's performance
 - Feature transformation: standardization, normalization, etc.
 - Feature creation: combine features, use domain knowledge, etc.
- Feature engineering is iterative
 - Need try-and-error loops

- Feature engineering can impact your model's performance
 - Feature transformation: standardization, normalization, etc.
 - Feature creation: combine features, use domain knowledge, etc.
- Feature engineering is iterative
 - Need try-and-error loops
- Ensembling is very useful in real-world applications
 - Bagging: random forest
 - Boosting: AdaBoost, XGBoost, LightGBM, etc.

